

Министерство цифрового развития, связи и массовых коммуникаций Российской
Федерации
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра ПМиК

Расчетно-графическая работа по дисциплине
«Защита информации»
на тему «Доказательство с нулевым знанием»
«Вариант 2»

Выполнил: студент 4 курса
ИВТ, гр. ИП-113
Шпилев Д. И.
Проверил: преподаватель кафедры ПМиК
Дьячкова Марина Сергеевна
.

Новосибирск 2024

Содержание

1. Постановка задачи	3
2. Теоретические сведения	4
3. Ход работы.....	9
4. Результат работы программы.....	12
5. Листинг кода.....	14

Постановка задачи

Необходимо написать программу, реализующую алгоритм доказательства с нулевым знанием «Гамильтонов цикл».

Обе рассматриваемые задачи являются NP-полными и не имеют быстрых методов для решения, поэтому для тестирования необходимо будет генерировать правильные решения при помощи дополнительно разработанных программ.

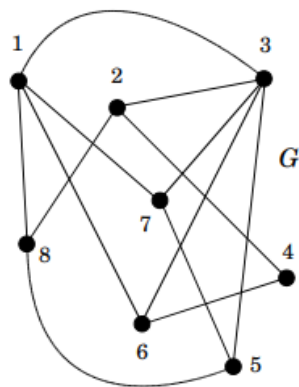
Вне зависимости от варианта задания, необходимо информацию о графах считывать из файла. В файле описание графа будет определяться следующим образом:

- 1) в первой строке файла содержатся два числа $n < 1001$ и $m \leq n^2$, количество вершин графа и количество рёбер соответственно;
- 2) в последующих m строках содержится информация о рёбрах графа, каждое из которых описывается с помощью двух чисел (номера вершин, соединяемых этим ребром);
- 3) в зависимости от варианта указывается необходимая дополнительная информация: в первом варианте перечисляются цвета вершин графа; во втором варианте указывается последовательность вершин, задающая гамильтонов цикл (этот пункт можно вынести в отдельный файл).

Теоретические сведения

Рассматриваемая в данном разделе задача не просто предоставляет нам возможность описать еще одну схему построения протокола доказательства с нулевым знанием, но и имеет важное теоретическое значение. Блум (Manuel Blum) показал, что, выражаясь неформально, любое математическое утверждение может быть представлено в виде графа, причем доказательство этого утверждения соответствует гамильтонову циклу в этом графе. Поэтому наличие протокола доказательства с нулевым знанием для гамильтонова цикла означает, что доказательство любого математического утверждения может быть представлено в форме доказательства с нулевым знанием.

Определение 5.1. Гамильтоновым циклом в графе называется непрерывный путь, проходящий через все вершины графа ровно по одному разу.



Путь, проходящий последовательно через вершины 8, 2, 4, 6, 3, 5, 7, 1, представляет собой гамильтонов цикл. Действительно, в этом Криптографические протоколы пути содержатся все вершины графа, и каждая вершина посещается только один раз. $\square$

Ясно, что если в графе G с n вершинами гамильтонов цикл существует, то при некоторой нумерации вершин он пройдет точно через вершины с последовательными номерами $1, 2, 3, \dots, n$. Поэтому путем перебора всех возможных нумераций вершин мы обязательно найдем гамильтонов цикл. Но количество возможных нумераций равно $n!$, и поэтому уже при умеренно больших n , например, при $n = 100$, такой подход становится практически нереализуемым. Доказано, что задача нахождения гамильтонова цикла в графе является NP-полной. Мы уже говорили кратко о понятии NP-полноты. Неформально, NP-полнота рассматриваемой задачи означает, что для ее решения не существуют (точнее, неизвестны) алгоритмы существенно более быстрые, чем указанный метод перебора.

Нашей задачей будет построение криптографического протокола, с помощью которого Алиса будет доказывать Бобу, что она знает гамильтонов цикл в некотором графе G так, чтобы Боб не получил никаких знаний о самом этом цикле. Иными словами, Алиса будет предоставлять Бобу доказательство с нулевым знанием. Еще раз напомним читателю, что «нулевое знание» означает, что независимо от числа

реализаций протокола доказательства Боб будет располагать точно такими же сведениями о гамильтоновом цикле, какие он мог бы получить, просто изучая представленный ему граф G .

Итак, допустим, что Алиса знает гамильтонов цикл в графе G . Теперь она может это доказывать Бобу и всем, кто имеет граф G , с помощью описываемого ниже протокола. Алиса может использовать это доказательство, например, для идентификации своей личности. Но прежде чем мы перейдем к описанию протокола, договоримся о некоторых обозначениях.

Мы будем обозначать графы буквами G, H, F , понимая под этим одновременно соответствующие матрицы смежности. Элемент матрицы $H_{ij} = 1$, если в графе H есть ребро, соединяющее вершины i и j ; $H_{ij} = 0$ в противном случае. Символом k будем обозначать конкатенацию (сцепление) двух чисел, точнее, двоичных слов, им соответствующих. Нам понадобится шифр с открытым ключом. Вообще говоря, это может быть любой шифр, но для определенности будем использовать шифр RSA. Будем считать, что Алиса сформировала систему RSA с открытыми параметрами N и d . Важно, что зашифрованные в этой системе сообщения может расшифровать только Алиса и больше никто.

Протокол доказательства состоит из следующих четырех шагов (пояснения будут даны ниже).

Шаг 1. Алиса строит граф H , являющийся копией исходного графа G , где у всех вершин новые, случайно выбранные номера. На языке теории графов говорят, что H изоморфен G . Иными словами, H получается путем некоторой перестановки вершин в графе G (с сохранением связей между вершинами). Алиса кодирует матрицу H , приписывая к первоначально содержащимся в ней нулям и единицам случайные числа r_{ij} по схеме $\tilde{H}_{ij} = r_{ij} \| H_{ij}$. Затем она шифрует элементы матрицы $\tilde{H}$, получая зашифрованную матрицу F , $F_{ij} = \tilde{H}_{ij}^d \bmod N$.

Матрицу F Алиса передает Бобу.

Шаг 2. Боб, получив зашифрованный граф F , задает Алисе один из двух вопросов.

1. Каков гамильтонов цикл для графа H ?
2. Действительно ли граф H изоморфен G ?

Шаг 3. Алиса отвечает на соответствующий вопрос Боба.

1. Она расшифровывает в F ребра, образующие гамильтонов цикл.
2. Она расшифровывает F полностью (фактически передает Бобу граф $\tilde{H}$) и предъявляет перестановки, с помощью которых граф H был получен из графа G .

Шаг 4. Получив ответ, Боб проверяет правильность расшифровки путем повторного шифрования и сравнения с F и убеждается либо в том, что показанные ребра

действительно образуют гамильтонов цикл, либо в том, что предъявленные перестановки действительно переводят граф G в граф H . Весь протокол повторяется t раз. Обсудим вначале кратко несколько вопросов по построению протокола.

1. Зачем Алиса строит изоморфный граф? Если бы она этого не делала, то Боб, получив ответ на свой вопрос номер один, узнал бы гамильтонов цикл в графе G .
2. Зачем Алиса кодирует матрицу H ? С этим приемом мы уже встречались при шифровании цветов вершин графа. Дело в том, что невозможно зашифровать непосредственно нули и единицы (с помощью шифра RSA они вообще не шифруются). Даже если заменить их на какие-то произвольные числа a и b , то мы получим всего два различных шифротекста, и Бобу не составит труда понять, какой из них какому числу соответствует. Т.е. структура графа не будет скрыта. Здесь мы сталкиваемся с типичной ситуацией, когда требуется использовать так называемый рандомизированный шифр. И такой шифр строится путем добавления случайных чисел в матрицу H перед шифрованием. Закодированная матрица H^* точно также задает граф (нечетность числа означает наличие ребра, четность — его отсутствие), но после шифрования H^* структура графа полностью скрывается (мы используем известное свойство шифра RSA — он полностью скрывает четность числа).
3. Зачем Боб задает два вопроса? Если бы он задавал только вопрос номер один, который по смыслу протокола является основным, то Алиса, не зная в действительности гамильтонова цикла в графе G , могла бы предъявить Бобу совсем другой граф с таким же количеством вершин и искусственно заложенным в него гамильтоновым циклом. Поэтому Боб иногда просит Алису доказать изоморфизм графов H и G . Важно, что Алиса не знает заранее, какой из двух вопросов задаст Боб.
4. Почему Боб не может задать сразу двух вопросов? В этом случае он узнал бы гамильтонов цикл в G , так как ему был бы показан гамильтонов цикл в H и правило перехода от H к G .
5. Зачем Боб проверяет правильность расшифровки? Если бы он этого не делал, то Алиса на четвертом шаге могла бы предоставить ему «выгодную» для себя информацию, а не ту, которую она посылала ему на втором шаге.

Более точно основные детали протокола обосновываются в ходе доказательства двух основных утверждений.

Д о к а з а т е л ь с т в о . Вначале покажем, что вероятность обмана в одной реализации протокола равна $1/2$. Заметим, что если Алиса действительно знает гамильтонов цикл в графе G , то она может правильно ответить на любой вопрос Боба. Если же она не знает гамильтонов цикл, то самое большее, что она может сделать, — это подготовиться к ответу на первый либо на второй вопрос. В ожидании первого вопроса, она создает новый граф с искусственно заложенным в него гамильтоновым циклом. Но в этом случае она не сможет доказать его изоморфизм графу G . В ожидании второго вопроса, она строит граф, изоморфный графу G . Но в этом случае

она не сможет показать в нем гамильтонов цикл. Таким образом, вероятность успешности обмана равна вероятности угадывания номера вопроса. В предположении, что Боб задает оба вопроса с одинаковыми вероятностями, мы получаем, что вероятность обмана равна.

Д о к а з а т е л ь с т в о . Чтобы доказать, что Боб не получает никаких знаний в ходе реализации протокола, достаточно показать, что все, что он получает от Алисы, он мог бы получить сам, не вступая с ней ни в какое общение.

Рассмотрим вначале второй вопрос Боба. В ответ на этот вопрос он получает граф, изоморфный графу G . Но он сам мог строить сколько угодно изоморфных графов, и то, что присылает ему Алиса, это просто один из них.

Случай с первым вопросом не столь очевиден. В ответ на первый вопрос Боб получает гамильтонов цикл в графе, изоморфном графу G . На первый взгляд может показаться, что это дает Бобу какую-то информацию. Однако это не так. Заметим, что если в G есть гамильтонов цикл, то при некоторой нумерации вершин существует

$$\begin{pmatrix} * & 1 & * & \dots & * & * & * \\ * & * & 1 & \dots & * & * & * \\ & & & \dots & & & \\ * & * & * & \dots & * & 1 & * \\ * & * & * & \dots & * & * & 1 \\ 1 & * & * & \dots & * & * & * \end{pmatrix},$$

где $*$ означает неопределенность в наличии или отсутствии ребра. Т.е. при такой нумерации гамильтонов цикл проходит через вершины в порядке возрастания номеров. Изменяя нумерацию вершин, Боб может получать из всевозможные изоморфные матрицы. Когда Алиса, отвечая на его первый вопрос, открывает гамильтонов цикл, Боб видит как раз одну из таких матриц. Таким образом, Боб не получает от Алисы никакой информации, которую он не мог бы получить сам. $\square$ Рассмотрим пример, иллюстрирующий все основные этапы описанного протокола.

П р и м е р. Возьмем в качестве основного графа G , изображенный на Его матрица смежности имеет вид

$$G = \begin{pmatrix} & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{matrix} & \begin{matrix} 0 & 0 & 1 & 0 & 0 & 1 & 1 & \boxed{1} \\ 0 & 0 & 1 & \boxed{1} & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & \boxed{1} & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & \boxed{1} & 1 \\ 1 & 0 & \boxed{1} & 1 & 0 & 0 & 0 & 0 \\ \boxed{1} & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & \boxed{1} & 0 & 0 & 1 & 0 & 0 & 0 \end{matrix} \end{pmatrix}.$$

В матрице с помощью $\cdot$ показан гамильтонов цикл. Алиса выбирает некоторую случайную нумерацию вершин, скажем, 7, 4, 5, 3, 1, 2, 8,

$$H = \begin{pmatrix} & 7 & 4 & 5 & 3 & 1 & 2 & 8 & 6 \\ 7 & 0 & 0 & 1 & 1 & \boxed{1} & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & \boxed{1} \\ 5 & \boxed{1} & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 3 & 1 & 0 & \boxed{1} & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & \boxed{1} & 1 \\ 2 & 0 & \boxed{1} & 0 & 1 & 0 & 0 & 1 & 0 \\ 8 & 0 & 0 & 1 & 0 & 1 & \boxed{1} & 0 & 0 \\ 6 & 0 & 1 & 0 & \boxed{1} & 1 & 0 & 0 & 0 \end{pmatrix}.$$

Для шифрования матрицы будем использовать систему RSA с параметрами $N = 55$, $d = 3$. Вначале закодируем матрицу H . В рамках данного примера просто припишем слева к каждому элементу матрицы выбираемую случайно с равными вероятностями цифру из множества $\{1, 2, 3, 4, 5\}$:

$$\tilde{H} = \begin{pmatrix} 50 & 20 & 11 & 31 & 21 & 40 & 20 & 10 \\ 40 & 30 & 50 & 20 & 10 & 41 & 50 & 21 \\ 41 & 30 & 50 & 11 & 30 & 20 & 51 & 40 \\ 11 & 10 & 41 & 30 & 51 & 41 & 30 & 21 \\ 31 & 20 & 40 & 11 & 50 & 10 & 41 & 31 \\ 50 & 41 & 20 & 21 & 40 & 10 & 21 & 50 \\ 40 & 30 & 31 & 50 & 41 & 21 & 30 & 40 \\ 20 & 41 & 10 & 51 & 41 & 20 & 30 & 40 \end{pmatrix}.$$

Теперь мы шифруем матрицу $\tilde{H}$, возводя каждый ее элемент в куб по модулю 55:

$$F = \begin{pmatrix} 40 & 25 & 11 & 36 & 21 & 35 & 25 & 10 \\ 35 & 50 & 40 & 25 & 10 & 06 & 40 & 21 \\ 06 & 50 & 40 & 11 & 50 & 25 & 46 & 35 \\ 11 & 10 & 06 & 50 & 46 & 06 & 50 & 21 \\ 36 & 25 & 35 & 11 & 40 & 10 & 06 & 36 \\ 40 & 06 & 25 & 21 & 35 & 10 & 21 & 40 \\ 35 & 50 & 36 & 40 & 06 & 21 & 50 & 35 \\ 25 & 06 & 10 & 46 & 06 & 25 & 50 & 35 \end{pmatrix}.$$

(При внимательном просмотре матрицы F может показаться, что использованный нами шифр плохо скрывает исходную матрицу. Криптографические протоколы Это объясняется тем, что, во-первых, модуль 55 слишком мал и, во вторых, в матрице $\tilde{H}$ много чисел, не взаимно простых с модулем.

Ход работы

Euclid – класс Алгоритма Евклида. Можно просчитать значения либо при создании элемента в конструкторе, либо вызвав метод `extendedGCD(long long a, long long b)`

long long inversion(long long c, long long m):

Инверсия числа c по модулю m .

bool isPrime(const long long &number, const int &accuracy = 10):

Функция проверяет число на простоту

long long genPrime(long long min, long long max):

Функция возвращает случайное простое число из передаваемой границы

long long genMutuallyPrime(const long long &P):

Функция возвращает случайное число взаимно-простое с передаваемым

long long modulPow(long long basis, long long indicator, const long long &modul):

Функция возводит в степень по модулю

RSA – класс при создании формирует ключи N , e , d . Получить данные можно через статический метод `getInstance()`.

Graph getGraphFromFile(const std::string& filePath):

Создает граф по исходному файлу

Graph – класс для работы с графом. Содержит следующие методы:

void addEdge(int a, int b) noexcept:

Добавление ребра между вершинами

std::vector<int> genPermutation():

Возвращает перемешанные индексы вершин

std::vector<int> genUnPermutation(const std::vector<int>& permutations):

Возвращает начальные позиции индексом относительно `permutations`

Graph makeIsomorph(const std::vector<int>& verices) const noexcept:

Возвращает изоморфный граф, параметром передается перестановка вершин

void randConcatenation():

Добавление случайных чисел в начало каждого значения матрицы смежности

Graph rsaEncode() const:

Кодирование матрицы смежности

std::vector<long long> hamCycle();

Данная функция ищет Гамильтонов цикл по графу.

Гамильтонов цикл — это цикл в графе, который посещает каждую вершину ровно один раз и возвращается в исходную вершину. Давайте рассмотрим, как работает этот метод:

1. Инициализация пути: Метод создает вектор `path` размером `V` (где `V` — количество вершин в графе), и инициализирует его значениями `-1`. Это будет представлять путь, который мы будем заполнять во время поиска цикла. Первая вершина в пути устанавливается равной `0` (это может быть начальная вершина).
2. Вызов вспомогательной функции: Метод вызывает вспомогательную функцию `hamCycleUtil(path, 1)`, которая будет пытаться заполнить путь, начиная со второй позиции (так как первая позиция уже заполнена).
3. Проверка результата: После вызова `hamCycleUtil`, если цикл не был найден (возвращает `false`), метод выводит сообщение о том, что такого решения не существует, и возвращает пустой вектор.
4. Завершение пути: Если цикл найден, метод добавляет начальную вершину (`0`) в конец вектора `path`, чтобы замкнуть цикл, и возвращает вектор `path`, в котором содержится последовательность вершин, представляющая найденный Гамильтонов цикл.

В целом, метод `hamCycle()` осуществляет поиск Гамильтонова цикла в графе, используя рекурсивный подход, и возвращает последовательность вершин, которая составляют этот цикл, если он существует. Если цикла не существует, он возвращает пустой вектор.

void clearRandom();

Очистка случайных значений полученных в методе `randConcatenation()`.

int main(int argc, char argv):**

Основная часть программы.

Алгоритм программы эмулирует общение двух людей.

Сначала функция получает значения количества вершин `N`, ребер `M`, граф `G` и его копию граф `H`.

Вторым шагом, по основному графу находится Гамильтонов цикл. Затем столбцы смежной матрицы переставляются и порядковые переставления записываются в список, получая Изоморфный граф.

Затем к каждому элементу матрицы дописывается число от `1` до `N` (случайное число на каждый элемент столбца матрицы одно и то же, но для каждого столбца уникально), чтобы каждый элемент можно было закодировать по алгоритму RSA. После кодирования, матрица готова к отправке Бобу.

Боб, получив матрицу, задает один из двух вопросов Алисе:

- 1) Каков Гамильтонов цикл для графа?
- 2) Действительно ли граф `H` изоморфен `G`

Выбрав первый вопрос, Алиса отправляет Бобу Гамильтонов цикл и матрицу H . Боб преобразует матрицу H , выбирая элемент по порядку из Гамильтонова цикла и проходя по алгоритму RSA. Полученную матрицу он сравнивает с той, что получил изначально, она же матрица F . Если они идентичны, то Боб проходит по матрице при помощи Гамильтонова цикла. Если это возможно, то алгоритм отработал верно.

Выбрав второй вопрос, Алиса отправляет Бобу матрицу H , которая еще не закодирована. Боб кодирует матрицу. Если эта матрица равна той, что он получил изначально, то Боб продолжает преобразовывать матрицу.

Следующим шагом он отбрасывает от каждого элемента матрицы все разряды, оставляя разряд единиц в качестве элемента матрицы. Следом Алиса отправляет ему список переставленных порядков матрицы и Боб переставляет столбцы матрицы в соответствии с этим списком. Тем самым Боб сможет убедиться, что матрицы идентичны.

Результат работы программы

START GRAPH:								
	1	2	3	4	5	6	7	8
1	0	0	1	0	0	1	1	1
2	0	0	1	1	0	0	0	1
3	1	1	0	0	1	1	1	0
4	0	1	0	0	0	1	0	0
5	0	0	1	0	0	0	1	1
6	1	0	1	1	0	0	0	0
7	1	0	1	0	1	0	0	0
8	1	1	0	0	1	0	0	0

-----ALICE-----

PERMUTATIONS:
1 7 4 2 8 5 6 3

ISOMORPH GRAPH:

	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	1	1
2	1	0	0	0	0	1	0	1
3	0	0	0	1	0	0	1	0
4	0	0	1	0	1	0	0	1
5	1	0	0	1	0	1	0	0
6	0	1	0	0	1	0	0	1
7	1	0	1	0	0	0	0	1
8	1	1	0	1	0	1	1	0

HIS CYCLE:
1 2 6 5 4 3 7 8 1

Рис.1 Заполнение первоначального графа, создание изоморфного графа Алисой и нахождение в нем Гамильтонова цикла

GRAPH AFTER RAND:								
	1	2	3	4	5	6	7	8
1	60	71	50	50	61	80	31	61
2	81	30	80	10	70	31	40	11
3	10	80	60	61	70	20	61	20
4	20	10	71	30	31	10	80	51
5	11	20	80	31	30	81	30	70
6	10	41	60	60	81	20	50	21
7	71	70	51	40	60	50	40	51
8	71	41	20	61	80	71	31	30

ENCODED RSA GRAPH FROM ALICE:										
	1	2	3	4	5	6	7	8		
1	361889170	335914140	227879773	227879773	279591078	80764799	332808881	279591078		
2	79185214	39704231	80764799	412785704	194698083	332808881	61588689	15923735		
3	412785704	80764799	361889170	279591078	194698083	176257540	279591078	176257540		
4	176257540	412785704	335914140	39704231	332808881	412785704	80764799	313683985		
5	15923735	176257540	80764799	332808881	39704231	79185214	39704231	194698083		
6	412785704	213868056	361889170	361889170	79185214	176257540	227879773	105471219		
7	335914140	194698083	313683985	61588689	361889170	227879773	61588689	313683985		
8	335914140	213868056	176257540	279591078	80764799	335914140	332808881	39704231		

Рис.2 Преобразование графа Алисой. Добавление Случайных чисел и кодировка

```

-----BOB-----
1. What is the Hamiltonian cycle for graph H?
2. Is the graph H really isomorphic to G?
1
H CYCLE:
{1, 2, 71}
{2, 6, 31}
{6, 5, 81}
{5, 4, 31}
{4, 3, 71}
{3, 7, 61}
{7, 8, 51}
{8, 1, 71}
Edges in the matrix are equal
Hamiltonian cycle exists

```

Рис.3 Первый вопрос Алисе от Боба

```

-----BOB-----
1. What is the Hamiltonian cycle for graph H?
2. Is the graph H really isomorphic to G?
2
Graph of Bob = F graph
Clear random numbers from graph:

```

	1	2	3	4	5	6	7	8
1	0	1	1	0	0	0	1	0
2	1	0	1	1	1	0	1	0
3	1	1	0	1	0	1	0	0
4	0	1	1	0	0	0	0	1
5	0	1	0	0	0	1	0	1
6	0	0	1	0	1	0	1	0
7	1	1	0	0	0	1	0	0
8	0	0	0	1	1	0	0	0

```

UNPERMUTATIONS:
3 5 2 8 7 4 1 6
NEW G GRAPH:

```

	1	2	3	4	5	6	7	8
1	0	0	1	0	0	1	1	1
2	0	0	1	1	0	0	0	1
3	1	1	0	0	1	1	1	0
4	0	1	0	0	0	1	0	0
5	0	0	1	0	0	0	1	1
6	1	0	1	1	0	0	0	0
7	1	0	1	0	1	0	0	0
8	1	1	0	0	1	0	0	0

```

New G = G

```

Рис.4 Второй вопрос Алисе

Листинг кода

crypto_lib.hpp

```
#ifndef CRYPTO_LIB_HPP
#define CRYPTO_LIB_HPP

class Euclid
{
private:
    long long m_a;
    long long m_b;
    long long m_x;
    long long m_y;
    long long m_gcd;

public:
    Euclid(long long a, long long b);

    long long a();
    long long b();
    long long x();
    long long y();
    long long gcd();

    void extendedGCD(long long a, long long b);
    void printEquation();
};

long long modulPow(long long basis, long long indicator, const long long& modul);

long long inversion(long long c, long long m);

long long diffieHellman();

long long giantBabyStep(const long long& a, const long long& p, const long long& y);

bool isPrime(const long long &number, const int &accuracy = 10);

long long genPrime(long long min, long long max);
long long genPrimePandQ();
long long genMutuallyPrime(const long long& P);

long long genPrimitiveRoot(const long long &P);

#endif //CRYPTO_LIB_HPP
```

crypto_lib.cpp

```
#include "crypto_lib.hpp"
#include <stdexcept>
#include <vector>
#include <iostream>
#include <random>
#include <unordered_map>
#include <algorithm>

static std::random_device rd;

static std::default_random_engine generator(rd());

Euclid::Euclid(long long a, long long b)
{
    if (a == 0 && b == 0)
        throw std::invalid_argument("Both numbers cannot be zero.");

    extendedGCD(a, b);
}

long long Euclid::a() { return m_a; }

long long Euclid::b() { return m_b; }

long long Euclid::x() { return m_x; }

long long Euclid::y() { return m_y; }

long long Euclid::gcd() { return m_gcd; }

void Euclid::extendedGCD(long long a, long long b)
{
    if (a < b)
        std::swap(a, b);

    m_a = a;
    m_b = b;

    std::vector<long long> u = {a, 1, 0};
    std::vector<long long> v = {b, 0, 1};
    std::vector<long long> t(3);
    while (v[0] != 0)
    {
        long long q = u[0] / v[0];

        t[0] = u[0] % v[0];
        t[1] = u[1] - v[1] * q;
        t[2] = u[2] - v[2] * q;

        std::copy(v.begin(), v.begin() + v.size(), u.begin());
    }
}
```

```

        std::copy(t.begin(), t.begin() + t.size(), v.begin());
    }
    m_gcd = u[0];
    m_x = u[1];
    m_y = u[2];
}

void Euclid::printEquation()
{
    std::cout << m_a << "*" << m_x << " + " << m_b << "*" << m_y << " = " << m_gcd << std::endl;
}

long long modulPow(long long basis, long long indicator, const long long &modul)
{
    if (basis < 0)
        throw std::invalid_argument("Basis must be a natural number!");
    if (indicator < 0)
        throw std::invalid_argument("Indicator must be a natural number!");
    if (modul <= 0)
        throw std::invalid_argument("Modul must be greater than zero!");

    long long result = 1;

    basis %= modul;

    while (indicator > 0)
    {
        if (indicator & 1)
        {
            result = (result * basis) % modul;
        }
        indicator >>= 1;
        basis = (basis * basis) % modul;
    }

    return result;
}

long long inversion(long long c, long long m)
{
    if (c > m)
        c %= m;
    Euclid euclid(m, c);
    if (euclid.gcd() != 1)
    {
        throw std::runtime_error("Numbers must be mutually simple!");
    }
    long long res = euclid.y();
    if (res < 0)
        res += m;
    return res;
}

```



```

}

bool miillerTest(long long d, const long long &n)
{
    int a = 2 + rand() % (n - 4);

    int x = modulPow(a, d, n);

    if (x == 1 || x == n - 1)
        return true;

    while (d != n - 1)
    {
        x = (x * x) % n;
        d *= 2;

        if (x == 1)
            return false;
        if (x == n - 1)
            return true;
    }

    return false;
}

bool isPrime(const long long &number, const int &accuracy)
{
    if (number <= 1 || number == 4)
        return false;
    if (number <= 3)
        return true;

    if (number % 2 == 0)
        return false;

    int d = number - 1;
    while (d % 2 == 0)
        d /= 2;

    for (int i = 0; i < accuracy; i++)
        if (!miillerTest(d, number))
            return false;

    return true;
}

long long genPrimePandQ()
{
    std::uniform_int_distribution<long long unsigned> distribution(100000, 1000000000);

    while (1)

```

```

{
    long long Q = distribution(generator);
    if (isPrime(Q))
    {
        long long P = 2 * Q + 1;
        if (isPrime(P))
            return P;
    }
}

long long genPrime(long long min, long long max)
{
    std::uniform_int_distribution<long long unsigned> distribution(min, max);

    while (1)
    {
        long long Q = distribution(generator);
        if (isPrime(Q))
        {
            return Q;
        }
    }
}

long long genMutuallyPrime(const long long& P)
{
    std::uniform_int_distribution<long long unsigned> distribution(100000, P-1);
    while(true)
    {
        long long number = distribution(generator);
        Euclid euclid(number, P);
        if (euclid.gcd() == 1) return number;
    }
}

long long genPrimitiveRoot(const long long &P)
{
    std::uniform_int_distribution<long long> distribution(2, P - 1);

    while (1)
    {
        long long g = distribution(generator);
        if (modulPow(g, P / 2, P) == 1)
            return g;
    }
}

long long diffieHellman()
{
    long long P = genPrimePandQ();

```

```

long long g = genPrimitiveRoot(P);
std::cout << "P = " << P << " | g = " << g << std::endl;
std::uniform_int_distribution<long long> distribution(1, P - 1);
long long Xa = distribution(generator);
long long Xb = distribution(generator);
std::cout << "Xa = " << Xa << " | Xb = " << Xb << std::endl;
long long Ya = modulPow(g, Xa, P);
long long Yb = modulPow(g, Xb, P);
std::cout << "Ya = " << Ya << " | Yb = " << Yb << std::endl;
long long Zab = modulPow(Yb, Xa, P);
long long Zba = modulPow(Ya, Xb, P);
std::cout << "Zab = " << Zab << " | Zba = " << Zba << std::endl;
return Zab;
}

long long giantBabyStep(const long long &a, const long long &p, const long long &y)
{
    long long m, k;
    m = k = sqrt(p) + 1;
    std::unordered_map<long long, long long> mp;
    long long num = y % p;
    mp[y] = 0;
    for (int i = 1; i < m; ++i)
    {
        num = (num * a) % p;
        mp[num] = i;
    }
    num = modulPow(a, m, p);
    if (mp.count(num))
    {
        return m - mp[num];
    }
    long long step = num;
    for (int i = 2; i <= k; ++i)
    {
        num = (num*step)%p;
        if (mp.count(num))
        {
            return m * i - mp[num];
        }
    }
    return -1;
}

```

main.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <random>
#include <sstream>
#include "crypto_lib.hpp"

class RSA{
public:
    static RSA *getInstance();

    RSA(RSA& other) = delete;
    RSA(RSA&& other) = delete;
    void operator=(const RSA &) = delete;
    void operator=(RSA&&) = delete;
    long long N() {return m_N;}
    long long d() {return m_d;}
    void regenerate();

private:
    RSA();
    static RSA* m_rsa;
    long long m_fi;
    long long m_c;
    long long m_N;
    long long m_d;
};

RSA::RSA()
{
    regenerate();
}

RSA *RSA::getInstance()
{
    if(m_rsa==nullptr){
        m_rsa = new RSA();
    }
    return m_rsa;
}

RSA* RSA::m_rsa= nullptr;

void RSA::regenerate()
{
    long long P = genPrime(1000, 50000);
    long long Q = genPrime(1000, 50000);
    m_N = P*Q;
    m_fi = (P-1)*(Q-1);
```

```

    m_d = genMutuallyPrime(m_fi);
    m_c = inversion(m_d, m_fi);
}

class Graph{
public:
    Graph(int vertices) noexcept;

    void addEdge(int a, int b) noexcept;
    void print() const noexcept;
    void print2() const noexcept;
    std::vector<int> genPermutation();
    std::vector<int> genUnPermutation(const std::vector<int>& permutations);
    Graph makeIsomorph(const std::vector<int>& verices) const noexcept;
    void randConcatenation();
    Graph rsaEncode() const;
    std::vector<long long> hamCycle();
    void clearRandom();
    int size() const noexcept {return V;}
    long long at(int row, int col) const {return adjacency[row][col];}

    bool operator==(const Graph& other);

private:
    RSA *rsa;
    int V;
    std::vector<std::vector<long long>> adjacency;
    void setAt(int row, int column, long long value);
    bool isSafe(int v, const std::vector<long long>& path, int pos);
    bool hamCycleUtil(std::vector<long long>& path, int pos);
};

Graph::Graph(int verices) noexcept
{
    V = verices;
    adjacency.resize(verices);
    for(auto& it : adjacency) it.resize(verices, 0);
    rsa = RSA::getInstance();
}

void Graph::addEdge(int a, int b) noexcept
{
    adjacency[a][b] = 1;
    adjacency[b][a] = 1;
}

void Graph::print() const noexcept
{
    std::cout << "\033[33m";
    for(int i = 0; i < V; ++i){
        std::cout << "\t" << i+1;
    }
}

```

```

    }
    for(int i = 0; i < V; ++i){
        std::cout << std::endl << "\033[33m" << i+1 << "\033[0m";
        for(int j = 0; j < V; ++j){
            std::cout << "\t" << adjacency[i][j];
        }
    }
    std::cout << std::endl << std::endl;
}

void Graph::print2() const noexcept
{
    for(int i = 0; i < V; ++i){
        for(int j = 0; j < V; ++j){
            std::cout << adjacency[i][j] << ", ";
        }
        std::cout << std::endl;
    }
}

std::vector<int> Graph::genPermutation()
{
    std::vector<int> verices(V);
    for(int i = 0; i < V; ++i){
        verices[i] = i;
    }
    auto rd = std::random_device {};
    auto rng = std::default_random_engine { rd() };
    std::shuffle(std::begin(verices), std::end(verices), rng);
    return verices;
}

std::vector<int> Graph::genUnPermutation(const std::vector<int> &permutations)
{
    std::vector<int> unpermutatins(V);
    for(int i = 0; i < V; ++i){
        unpermutatins[permutations[i]] = i;
    }
    return unpermutatins;
}

Graph Graph::makeIsomorph(const std::vector<int>& verices) const noexcept
{
    Graph newGraph(V);
    for(int i = 0; i < V; ++i){
        for(int j = i; j < V; ++j){
            newGraph.setAt(i, j, adjacency[verices[i]][verices[j]]);
            newGraph.setAt(j, i, adjacency[verices[i]][verices[j]]);
        }
    }
    return newGraph;
}

```

```

}

void Graph::randConcatenation()
{
    auto rd = std::random_device {};
    auto rng = std::default_random_engine { rd() };
    std::uniform_int_distribution<int> distribution(1, V);
    std::ostringstream oss;
    for(auto& row : adjacency){
        for(auto& number : row){
            oss << distribution(rng) << number;
            std::istringstream(oss.str()) >> number;
            oss.str("");
        }
    }
}

Graph Graph::rsaEncode() const
{
    Graph newGraph(V);
    for(int i = 0; i < V; ++i){
        for(int j = 0; j < V; ++j){
            newGraph.setAt(i, j, modulPow(adjacency[i][j], rsa->d(), rsa->N()));
        }
    }
    return newGraph;
}

void Graph::setAt(int row, int column, long long value)
{
    adjacency[row][column] = value;
}

Graph Graph::getGraphFromFile(const std::string& filePath){
    std::ifstream in(filePath);
    int n, m;
    in >> n >> m;

    int a,b;
    Graph graph(n);
    while(m--){
        in >> a >> b;
        graph.addEdge(a-1,b-1);
    }
    return graph;
}

bool Graph::isSafe(int v, const std::vector<long long>& path, int pos)
{
    if (adjacency[path[pos - 1]][v] == 0)
        return false;

```

```

    for (int i = 0; i < pos; i++)
        if (path[i] == v)
            return false;

    return true;
}

bool Graph::hamCycleUtil(std::vector<long long>& path, int pos)
{
    if (pos == V)
    {
        if (adjacency[path[pos - 1]][path[0]] == 1)
            return true;
        else
            return false;
    }

    for (int v = 1; v < V; v++)
    {
        if (isSafe(v, path, pos))
        {
            path[pos] = v;

            if (hamCycleUtil(path, pos + 1) == true)
                return true;

            path[pos] = -1;
        }
    }

    return false;
}

std::vector<long long> Graph::hamCycle()
{
    std::vector<long long> path(V, -1);

    path[0] = 0;

    if (hamCycleUtil(path, 1) == false )
    {
        std::cout << "\nSolution does not exist";
        return std::vector<long long>();
    }
    path.push_back(0);
    return path;
}

void Graph::clearRandom()
{

```



```

for(auto& row : adjacency){
    for(auto& it : row){
        it&=1;
    }
}

bool Graph::operator==(const Graph &other)
{
    for(int i = 0; i < V; ++i){
        for(int j = 0; j < V; ++j){
            if (this->adjacency[i][j] != other.adjacency[i][j])
                return false;
        }
    }
    return true;
}

int main(int argc, char** argv)
{
    if(argc != 2) return -1;
    RSA *rsa = nullptr;
    Graph G = getGraphFromFile(std::string(argv[1]));
    std::cout << "START GRAPH:" << std::endl;
    G.print();
    std::cout << "-----ALICE-----" << std::endl;
    std::vector<int> permutation = G.genPermutation();
    std::cout << "PERMUTATIONS:" << std::endl;
    for(auto& it : permutation) std::cout << it+1 << " ";
    std::cout << std::endl;
    Graph H = G.makeIsomorph(permutation);
    std::cout << "ISOMORPH GRAPH:" << std::endl;
    H.print();
    std::cout << "HIS CYCLE:" << std::endl;
    std::vector<long long> cycle = H.hamCycle();
    for(auto& it : cycle) std::cout << it+1 << " ";
    std::cout << std::endl;
    H.randConcatenation();
    std::cout << "GRAPH AFTER RAND:" << std::endl;
    H.print();
    Graph F = H.rsaEncode();
    std::cout << "ENCODED RSA GRAPH FROM ALICE:" << std::endl;
    F.print();
    std::cout << "-----BOB-----" << std::endl;
    std::cout << "1. What is the Hamiltonian cycle for graph H?" << std::endl;
    std::cout << "2. Is the graph H really isomorphic to G?" << std::endl;
    int result;
    std::cin >> result;
    if (result == 1){
        std::vector<std::tuple<int, int, long long>> coords(F.size());
        int row, column;

```

```

long long number;
int pos = 0;
std::cout << "H CYCLE:" << std::endl;
for(int i = 0; i < F.size(); ++i){
    coords[i] = {pos, cycle[i+1], H.at(pos, cycle[i+1])};
    pos = cycle[i+1];
}
for(int i = 0; i < F.size(); ++i){
    std::tie(row, column, number) = coords[i];
    std::cout << "{" << row+1 << ", " << column+1 << ", " << number << "}" << std::endl;
}
bool flag = true;
rsa = RSA::getInstance();
for(int i = 0; i < F.size(); ++i){
    std::tie(row, column, number) = coords[i];
    if (modulPow(number, rsa->d(), rsa->N()) != F.at(row, column)){
        flag = false;
        break;
    }
}
if (flag){
    std::cout << "Edges in the matrix are equal" << std::endl;
    std::vector<bool> checkCycle(F.size(), false);
    for(int i = 0; i < coords.size(); ++i){
        std::tie(row, column, number) = coords[i];
        if (checkCycle[row]){
            flag = false;
            break;
        }
        checkCycle[row] = true;
    }
    if (flag){
        std::cout << "Hamiltonian cycle exists" << std::endl;
    }
    else{
        std::cout << "Hamiltonian cycle not exists" << std::endl;
    }
}
else{
    std::cout << "Edges in the matrix not equal" << std::endl;
}
}
else if (result == 2){
    Graph bobGraph = H.rsaEncode();
    if (bobGraph == F){
        std::cout << "Graph of Bob = F graph" << std::endl;
        H.clearRandom();
        std::cout << "Clear random numbers from graph:" << std::endl;
        H.print();
        std::vector<int> unpermutations = H.genUnPermutation(permutation);

```

```

std::cout << "UNPERMUTATIONS:" << std::endl;
for(auto& it : unpermutations) std::cout << it+1 << " ";
std::cout << std::endl;
std::cout << "NEW G GRAPH:" << std::endl;
Graph newG = H.makeIsomorph(unpermutations);
newG.print();
if (newG == G){
    std::cout << "New G = G" << std::endl;
}
else{
    std::cout << "New G != G" << std::endl;
}
}
else {
    std::cout << "Graph of Bob != F graph" << std::endl;
}
}
delete rsa;
return 0;
}

```